AfriOS

Composants par couche

Référence détaillée des briques techniques

Pour chaque couche d'AfriOS (1-6) : liste exhaustive des composants, leur rôle, leurs fichiers sources, leurs dépendances amont/aval, et l'API qu'ils exposent. 50+ composants documentés.

VERSION
1.0

DATE
26 September 2026

AUTEUR
Équipe AfriOS
Document technique AfriOS — github.com/ErdisKodjo/System

STATUT
Public

Table des matières

Table des matières 2
1. Introduction 4
1.1 Conventions . . . 4
1.2 Position dans la documentation . . . 4
1.3 Synthèse des composants par couche . . . 4
2. Couche 1 — Matériel 6
2.1 ARM64 (AArch64) . . . 6
2.2 x86_64 . . . 6
2.3 RISC-V 64 . . . 6
2.4 MCU Cortex-M . . . 7
2.5 Host mock (tests CI) . . . 7
3. Couche 2 — Firmware UEFI/EDK2 8
3.1 Vue d'ensemble . . . 8
3.2 Composants — Phases de boot . . . 8
3.3 Composants — BootManager . . . 8
3.4 Composants — Security . . . 9
3.5 Composants — OTA Update . . . 9
3.6 Composants — Diagnostics . . . 9
3.7 Composants — HAL firmware . . . 9
3.8 Composants — ShimLayer + SetupUi . . . 10
3.9 Composants — BoardConfigs . . . 10
3.10 API exposée pour la couche 3 (noyau) . . . 10
4. Couche 3 — Noyau afros-core 11
4.1 Composants — HAL (interfaces abstraites) . . . 11
4.2 Composants — HAL (implémentation commune) . . . 11
4.3 Composants — Ports HAL (5 ports) . . . 11
4.4 Composants — Sous-système scheduler . . . 12
4.5 Composants — Sous-système memory . . . 12
4.6 Composants — Sous-système network . . . 12
4.7 Composants — Sous-système power . . . 13
4.8 Composants — FS + Security + Drivers + Hypervisor . . . 13
4.9 API exposée pour la couche 4 (orchestrateur) . . . 13

5. Couche 4 — Orchestrateur corebridge 14
5.1 Composants — Loader . . . 14
5.2 Composants — Runtime managers (5) . . . 14
5.3 Composants — Unified execution . . . 14
5.4 Composants — Version management . . . 14
5.5 Composants — Selection + Monitoring . . . 15
5.6 API publique gelée v1.0.0 . . . 15
6. Couche 5 — Runtimes de compatibilité 16
6.1 Module afros-winbridge (Wine) . . . 16
6.2 Module afros-androsandbox (Android) . . . 16
6.3 Module afros-incompat-engine (iOS/macOS via Darling) . . . 17
6.4 Module afros-harmonygate (HarmonyOS) . . . 17
6.5 Runtime Linux natif . . . 18
7. Couche 6 — Graphismes (DXVK) 19
7.1 Composants — D3D9 (DirectX 9) . . . 19
7.2 Composants — D3D11 (DirectX 11) . . . 19
7.3 Composants — D3D12 (DirectX 12) . . . 19
7.4 Composants — DXGI + HLSL + Vulkan + Utils . . . 19
7.5 Points d'entrée Wine . . . 20
8. Vue transversale des interconnexions 21
8.1 Scénario : lancement de notepad.exe . . . 21
8.2 Composants impliqués par couche . . . 21
8.3 Dépendances critiques amont/aval . . . 21
9. Conclusion 23

1. Introduction

Ce document fait le pont entre le document 2 Nouvelle architecture (qui décrit les 6 couches à un niveau vision) et le document 3 Modules (qui détaille chaque module fichier par fichier). Ici, nous adoptons une granularité intermédiaire : pour chaque couche, nous listons tous les composants — une brique technique cohérente avec un rôle unique, des fichiers sources identifiés, des dépendances amont et aval, et une API exposée pour la couche supérieure.

1.1 Conventions

Chaque section couche suit la même structure : (1) vue d'ensemble de la couche et de ses composants principaux ; (2) tableau des composants avec colonnes Nom, Rôle, Fichiers, Dépend de ; (3) schéma d'organisation interne ; (4) API exposée pour la couche supérieure. Les numéros entre parenthèses (ex. (2.3)) renvoient aux sections des autres documents pour approfondir.

1.2 Position dans la documentation

AfriOS dispose désormais d'une documentation complète en 5 volumes + 1 white paper. Le tableau ci-dessous clarifie le périmètre de chacun.

Document	Granularité	Question répondue
1. Présentation implémentation	Mission	Quoi a été fait ?
2. Nouvelle architecture	Vision	Pourquoi ces couches ?
3. Modules	Fichiers	Quels fichiers dans tel module ?
4. Guide build complet	Procédure	Comment builder ?
5. Composants par couche (ce doc)	Briques	Quels composants ?
White paper optimisations	Algorithmes	Comment optimisent-ils ?

1.3 Synthèse des composants par couche

Couche	Composants	Volume total
1. Matériel	4 architectures × ~5 composants = ~20	~15 KB de headers
2. Firmware UEFI	~25 composants (SEC, PEI, DXE, BootManager, OVMF, libs, HAL, etc.)	~74 000 lignes
3. Noyau afros-core	~30 composants (HAL + ports + 6 sous-systèmes noyau)	~3 000 lignes
4. Orchestrateur corebridge	~20 composants (loader, 5 runtime mgrs, unified, services mgmt, monitoring)	5 139 lignes
5. Runtimes compat (5 modules)	~50 composants (PE loader, binder, ART, Mach-O, libs, etc.)	~31 000 lignes
6. Graphismes (DXVK)	~25 composants (D3D9/11/12, DXGI, HLSL, Vulkan)	5 542 lignes

Couche	Composants	Volume total
Total	~150 composants	~51 000 lignes

2. Couche 1 — Matériel

La couche 1 regroupe les composants matériels physiques sur lesquels AfriOS s'exécute. Pour chaque architecture supportée, le matériel fournit 6 services attendus par la HAL : CPU, mémoire, interruptions, timer, console (UART), et stockage (block device). Les composants ci-dessous ne sont pas du code AfriOS mais les briques hardware que la HAL doit piloter.

2.1 ARM64 (AArch64)

Architecture principale d'AfriOS, optimisée pour le terrain africain (SBC bon marché, serveurs Ampere Altra). Utilise la topologie big.LITTLE et le GIC v3 pour les interruptions.

Composant	Rôle	Spécificités AfriOS
CPU big (A72/A78)	Calcul haute performance	Placé en priorité pour CPU-bound
CPU LITTLE (A53/A55)	Calcul économe	Placé en priorité pour IO-bound
GIC v3	Contrôleur interruptions	Support IPI multi-cœur
MMU v8	Gestion mémoire virtuelle	Stage-2 pour hyperviseur
PL011 UART	Console série	Base 0x9000000 (QEMU virt)
Generic Timer	Compteurs arch (cntpct_el0)	Fréquence ~62.5 MHz typique

2.2 x86_64

Architecture desktop/serveur. Utilise APIC/IOAPIC pour les interruptions, VGA text mode pour la console, et le TSC comme timer haute résolution.

Composant	Rôle	Spécificités AfriOS
CPU Intel/AMD	Calcul symétrique	Mono-cluster, pas de big.LITTLE
APIC + IOAPIC	Contrôleur interruptions	IPI via LAPIC, IRQs via IOAPIC
MMU long mode	Gestion mémoire 4 niveaux	Stage-2 via EPT pour hyperviseur
VGA text mode	Console 80×25 chars	Mémoire à 0xB8000, 7 attributs couleur
16550 UART COM1	Console série fallback	Port I/O 0x3F8
RDTSC + HPET	Timer haute résolution	TSC pour ticks, HPET pour oneshot

2.3 RISC-V 64

Architecture émergente pour SBC et IoT. Utilise SBI (Supervisor Binary Interface) comme firmware bas niveau, PLIC pour les interruptions, et l'instruction rdtime pour le timer.

Composant	Rôle	Spécificités AfriOS
CPU RISC-V 64 (RV64GC)	Calcul	S-mode, XLEN=64
PLIC	Contrôleur interruptions	Pas d'IPI natif (utilise CLINT soft IRQ)
MMU Sv39/Sv48	Gestion mémoire	Stage-2 via HGATP pour hyperviseur
SBI console	Console via ecall	EID 0x01 = putchar, 0x02 = getchar
SBI timer	rdtime CSR	Fréquence = timebase-frequency DT
DT (Device Tree)	Découverte hardware	Passé par bootloader dans a0/a1

2.4 MCU Cortex-M

Architecture microcontrôleur pour cibles embarquées (STM32, nRF52, RP2040). Mono-cœur, pas de MMU (uniquement MPU), pas de mode supervisor/user (uniquement Thread/Handler mode).

Composant	Rôle	Spécificités AfriOS
CPU Cortex-M4/M7	Calcul temps réel	Thumb-2 ISA, pas de cache L1
NVIC	Contrôleur interruptions	3-4 bits de priorité, pas d'IPI
MPU (pas de MMU)	Protection régions mémoire	8 régions configurables
SysTick	Timer 24 bits	Sur AHB, fréquence = HCLK/8
USART STM32-style	Console série	Base 0x40011000 (USART1 STM32F1)
Flash + SRAM	Stockage + mémoire	Flash à 0x08000000, SRAM à 0x20000000

2.5 Host mock (tests CI)

Pseudo-architecture 5e qui fournit une implémentation userspace-safe de la HAL pour exécuter les tests sur Linux host sans QEMU. Critique pour la CI/CD.

Composant	Rôle	Implémentation
CPU mock	Calcul simulé	get_info retourne valeurs mock (2400 MHz)
Malloc	Mémoire	posix_memalign + free + mmap
Pas d'interruptions	Mock no-op	Toutes les ops retournent SUCCESS
clock_gettime	Timer	CLOCK_MONOTONIC, 1 GHz
printf	Console	fputc + select non-blocking
Backing file	Stockage	/tmp/afros-host-mock-storage.img (1 MB)

3. Couche 2 — Firmware UEFI/EDK2

Le firmware AfriOS est une platform package EDK2 (TianoCore) nommée HybridFirmwarePlatformPkg. Il implémente le boot UEFI multi-architectures avec extensions hybrides. Cette couche contient ~25 composants répartis en 9 sous-systèmes.

3.1 Vue d'ensemble

```
FirmwareHybride/edk2/HybridFirmwarePlatformPkg/ | +-- PlatformInit/ Phases SEC, PEI, DXE +-- BootManager/ Politique + 7 boot managers + menu + scripts +-- Security/ Secure Boot + Measured Boot (TPM) +-- OtaUpdate/ Slots A/B + agent capsule +-- Diagnostics/ POST + Shell extensions +-- ShimLayer/ Hyperviseur minimal + passthrough +-- HAL/ CPU, PlatformDetect, ACPI, SMBIOS, DeviceTree +-- SetupUi/ Formulaires HII VFR + strings .uni +-- Libraries/ TimerLib (multi-arch) +-- BoardConfigs/ rpi4, pine64, qemu-virt, qemu-virt-riscv
```

3.2 Composants — Phases de boot

Composant	Phase UEFI	Rôle	Fichiers
MultiArchResetVector.S	SEC	Reset vector multi-arch	PlatformInit/Sec/
MemoryInit.c	PEI	Init RAM + MMU PEI	PlatformInit/Pei/
PlatformInfoPei.c	PEI	Publie HOB FDT/ACPI	PlatformInit/Pei/
PlatformInitDxe.c	DXE	Init plateforme runtime	PlatformInit/Dxe/
AcpiPlatformDxe.c	DXE	Installe tables ACPI	PlatformInit/Dxe/

3.3 Composants — BootManager

Composant	Rôle	Fichiers
BootPolicyEngine.c	Choisit entrée de boot (priority + fallback)	BootManager/
LinuxBootManager.c	Boot Linux via EFI Stub ou GRUB	BootManager/UefiBootManager/
GenericBootManager.c	Boot UEFI générique (load + start image)	BootManager/UefiBootManager/
WinBootMgr.c	Localise et démarre bootmgfw.efi	BootManager/WindowsBoot/
AppleBootHelper.c	Démarre boot.efi avec boot args	BootManager/AppleBoot/
ConfigPlistParser.c	Parse XML config.plist (OpenCore/Clover)	BootManager/AppleBoot/
IpxeDriver.c	Driver binding EFI pour boot réseau iPXE	BootManager/PxeBoot/
HttpBootClient.c	Client HTTP Boot (DHCP opt 60 + HTTP GET)	BootManager/PxeBoot/
BootMenuUi.c	Menu texte multi-OS (navigation clavier)	BootManager/BootMenu/
BootScripts.c	Moteur scripts .nsh-like	BootManager/BootMenu/
CsmPolicy.c	Politique repli BIOS legacy (CSM)	BootManager/LegacyCsm/

3.4 Composants — Security

Composant	Rôle	Fichiers
SecureBootPolicy.c	Vérifie signatures modules UEFI au chargement	Security/SecureBoot/
MeasuredBootLib.c	Étend mesures dans PCR TPM (0,2,4,7,11)	Security/MeasuredBoot/

3.5 Composants — OTA Update

Composant	Rôle	Fichiers
ABSlotManager.c	Gère slots A/B + rollback NVRAM (variable AfriBootNext)	OtaUpdate/
FwUpdateAgent.c	Lit capsule UEFI, valide header, appelle UpdateCapsule	OtaUpdate/
CapsuleEngine.c	Traitement capsules (parsing + dispatch)	OtaUpdate/

3.6 Composants — Diagnostics

Composant	Rôle	Fichiers
MemoryTest.c	POST mémoire (motif 0x55AA55AA)	Diagnostics/PowerOnSelfTest/
PciEnumTest.c	Énumération PCI (vendor/device/class/BARs)	Diagnostics/PowerOnSelfTest/
ShellExtensions.c	Commande afri (ver/slot/capsule/pci/memtest)	Diagnostics/UefiShell/
DebugScripts.nsh	Scripts .nsh appelant afri_*	Diagnostics/UefiShell/

3.7 Composants — HAL firmware

Composant	Rôle	Fichiers
CpuHalLib.c	Topologie CPU multi-arch (MDE_CPU_* macros)	HardwareAbstractionLayer/CpuHa
PlatformDetectLib.c	Détection plateforme (ACPI vs FDT)	HardwareAbstractionLayer/Platfo
AcpiTableGenerator.c	Génère FADT + DSDT dynamiquement	HardwareAbstractionLayer/Acpi/
Fadt.c	Table FADT (Fixed ACPI Description Table)	HardwareAbstractionLayer/Acpi/A
Dsdt.asl	DSDT ASL (CPU, HPET, RTC, device AfriOS)	HardwareAbstractionLayer/Acpi/A
SmbiosGenerator.c	Génère tables SMBIOS	HardwareAbstractionLayer/Smbio
FdtPlatformDxe.c	Driver DXE pour Device Tree	HardwareAbstractionLayer/Device
TimerLib.c	TimerLib multi-arch (RDTSC/cntpct_el0/rdtime)	Libraries/TimerLib/

3.8 Composants — ShimLayer + SetupUi

Composant	Rôle	Fichiers
ShimLoader.c	Loader pour la shim layer (transitions UEFI→AfriOS)	ShimLayer/
VmcsInit.c	Init VMCS pour VT-x (hyperviseur minimal)	ShimLayer/MinimalHypervisor/
Passthrough.c	Passthrough EPT/Stage-2 pour accès 1:1 périphériques	ShimLayer/MinimalHypervisor/
MainForm.vfr	Formulaire HII principal (setup)	SetupUi/HiiForms/
BootOrderForm.vfr	Formulaire HII boot order + secure boot	SetupUi/HiiForms/
BootOrderForm.uni	Strings HII bilingues FR/EN	SetupUi/HiiForms/

3.9 Composants — BoardConfigs

Composant	Carte cible	PcdFdtBaseAddr
Rpi4.dsc.include	Raspberry Pi 4 (ARM64 Cortex-A72)	0x01000000
Pine64.dsc.include	Pine64 A64 (ARM64 Cortex-A53)	0x40000000
QemuVirt.dsc.include	QEMU virt ARM64	0x40000000
QemuVirtRiscv.dsc.include	QEMU virt RISC-V 64	0x87E00000

3.10 API exposée pour la couche 3 (noyau)

Le firmware expose son API via : (a) des protocoles UEFI standards (EFI_BOOT_MANAGER_LOAD_OPTION, EFI_PCI_IO_PROTOCOL, EFI_SIMPLE_TEXT_INPUT_PROTOCOL, etc.) ; (b) des variables NVRAM AfriOS (AfriBootInfo, AfriCsmPolicy, AfriFwUpdateState) ; (c) des tables ACPI et SMBIOS installées en DXE ; (d) le HOB FDT publié par PlatformInfoPei.c pour utilisation par le noyau.

4. Couche 3 — Noyau afros-core

Le noyau AfriOS est structuré en 4 sous-ensembles : (a) HAL (interfaces abstraites + implémentation commune), (b) ports (implémentations par architecture), (c) sous-systèmes noyau génériques (scheduler, memory, network, power, fs, security), (d) drivers et hypervisor. Cette couche contient ~30 composants.

4.1 Composants — HAL (interfaces abstraites)

Composant	Rôle	Fichiers
cpu_abstraction.h	Interface CPU (get_info, set_frequency, sleep/wakeup, halt/reset)	hal/include/
memory_abstraction.h	Interface mémoire (alloc/free/map/unmap/compress)	hal/include/
interrupt_abstraction.h	Interface IRQ (enable/disable/register_handler/ack/send_ipi)	hal/include/
timer_abstraction.h	Interface timer (get_ticks/set_oneshot/set_periodic/busy_wait_us)	hal/include/
console_abstraction.h	Interface console (putc/puts/getc/has_input)	hal/include/
storage_abstraction.h	Interface stockage (read_blocks/write_blocks/flush)	hal/include/
device_abstraction.h	Interface device générique (register/read/write/ioctl)	hal/include/
afros_types.h	Types communs (status_t, cpu_info_t, task_t, etc.)	hal/include/
afros_hal.h	Header umbrella + hal_ops_t (init/reset/shutdown/suspend/resume)	hal/include/
power_manager.h	Interface gestion énergie (states, events, AC detection)	hal/include/

4.2 Composants — HAL (implémentation commune)

Composant	Rôle	Fichiers
hal_init.c	Agrège les op-tables en hal_ops_t + init sequence	hal/src/
power_manager.c	Implémente power_manager.h (states + events)	hal/src/
device_manager.c	Registre devices + dispatch read/write/ioctl	hal/src/
io_subsystem.c	File IRPs (I/O Request Packets) + completion routines	hal/src/
gpu_manager.c	Gestion GPU (placeholder, à brancher sur DRM	hal/src/
network_manager.c	Coordonne interfaces réseau	hal/src/
virtualization.c	Interface hyperviseur (alloc vmcs, map guest)	hal/src/
kprintf.c	Formateur freestanding (%s%d%x%c) sans libc	hal/src/

4.3 Composants — Ports HAL (5 ports)

Port	Console	Timer	Specificités
port-arm64	PL011 UART @0x9000000	cntpct_el0 + CNTV_CTL	big.LITTLE + GIC v3
port-x86_64	VGA text + 16550 COM1	RDTSC + HPET	APIC + IOAPIC
port-riscv	SBI console (EID 0x01)	rdtime CSR	S-mode + PLIC, vector_table.S S-mode
port-mcu	USART STM32-style	SysTick	NVIC, vector_table.S Cortex-M, crt0.c, linker-m
port-host-mock	printf	clock_gettime(CLOCK_MONOTONIC)	hosted Linux QEMU

Chaque port fournit 6 fichiers _port.c (console, cpu, memory, interrupt, timer, storage) qui implémentent les op-tables attendues par la HAL. Le port actif est sélectionné via la variable CMake AFROS_PORT.

4.4 Composants — Sous-système scheduler

Composant	Rôle	Fichiers
afros_cfs.c	Completely Fair Scheduler (VRuntime, red-black tree)	afros/scheduler/
afros_scheduler.c	Wrapper orchestrateur scheduler (init/run/yield)	afros/scheduler/
big_little.c	Détection topologie + placement initial	afros/scheduler/
big_little_support.c	EWMA load + hystérésis 3 + solar boost	afros/scheduler/
power_aware.c	Admission control par classe (CRITICAL/USER/BG/BATCH)	afros/scheduler/
power_aware_sched.c	File prête multi-classe + gel/dégel par batterie	afros/scheduler/
task_migration.c	Migration cross-CPU (load balancing)	afros/scheduler/

4.5 Composants — Sous-système memory

Composant	Rôle	Fichiers
adaptive_reclaim.c	Récupération pages peu utilisées (LRU)	afros/memory/
adaptive_swap.c	Swap adaptatif (pas de swap en mode solaire)	afros/memory/
compression.c	Compression pages avant swap	afros/memory/
numa_aware.c	Détection topologie NUMA multi-socket	afros/memory/
numa_balancer.c	Migration pages vers home node (seuil 70%)	afros/memory/

4.6 Composants — Sous-système network

Composant	Rôle	Fichiers
tcp_westwood_africa.c	TCP Westwood+ Africa (filtre alpha=3, lien intermittent)	afros/network/

Composant	Rôle	Fichiers
intermittent_net.c	DTN-lite (file RAM 1024, priorisation CRITICAL/NORMAL/BULK)	afros/network/
protocol_compression.c	ROHC RFC 3095 simplified (64 contexts)	afros/network/
tcp_acceleration.c	Accélération TCP (TSO, GRO)	afros/network/
intelligent_routing.c	Routage intelligent (multi-path, QoS)	afros/network/
protocol_optimization.c	Optimisations protocoles (Nagle off, ACK piggybacking)	afros/network/

4.7 Composants — Sous-système power

Composant	Rôle	Fichiers
solar_aware.c	Détection source solaire + boost CPU	afros/power/
opportunistic_compute.c	Fenêtres solaires pour tâches BATCH	afros/power/
opportunistic_sleep.c	Sleep opportuniste (cœurs inactifs)	afros/power/
brownout_protection.c	Protection contre brownouts (chute tension)	afros/power/

4.8 Composants — FS + Security + Drivers + Hypervisor

Composant	Rôle	Fichiers
afrosfs/journal.c	Journaling AfrosFS (write-ahead log)	afros/fs/afrosfs/
afrosfs/file.c	Opérations fichier (open/read/write/close)	afros/fs/afrosfs/
security/secure_boot.c	Vérification kernel-side secure boot	afros/security/
pci/afros_pci.c	Driver PCI (énumération + BAR mapping)	drivers/pci/
bootmanager/boot_manager.c	Boot manager kernel-side (NVRAM entries)	Kernel/bootmanager/
hypervisor.h	Interface hyperviseur minimal	Kernel/hypervisor/

4.9 API exposée pour la couche 4 (orchestrateur)

```
// HAL ops (fournies par le port actif) extern hal_ops_t afros_hal_ops; extern cpu_ops_t arch_cpu_ops; extern memory_ops_t arch_memory_ops; extern interrupt_ops_t arch_interrupt_ops; extern timer_ops_t arch_timer_ops; extern console_ops_t arch_console_ops; extern storage_ops_t arch_storage_ops; // Entry point noyau void kernel_main(void); // Logging freestanding int kprintf(const char *fmt, ...); #define afros_log_info(fmt, ...) kprintf("[INFO]" fmt, ##__VA_ARGS__)
```

5. Couche 4 — Orchestrateur corebridge

L'orchestrateur est le cœur fonctionnel d'AfriOS. Sans lui, les 5 runtimes sont des îlots isolés ; avec lui, ils deviennent un système unifié. Contient ~20 composants répartis en 5 sous-systèmes : loader, runtime managers, unified execution, version management, et selection/monitoring.

5.1 Composants — Loader

Composant	Rôle	Lignes
app_detector.c	Détection format par magic bytes (PE/ELF/Mach-O/DEX/HAP)	161
format_analyzer.c	Analyse détails (PE subsystem, ELF interp, Mach-O arch, DEX version, HarmonyOS mo	410
intelligent_loader.c	Décision finale + cache LRU + budget filter	252
dependency_resolver.c	Résolution dépendances par type (PE imports, DT_NEEDED, LOAD_DYLIB, dex2oa	298

5.2 Composants — Runtime managers (5)

Composant	Runtime	Backend wrappé
linux_runtime_manager.c	Linux natif	clone() + CLONE_NEWNS|NEWPID
win_runtime_manager.c	Windows	wineserver + wine_loader
android_runtime_manager.cpp	Android	service_manager + surface_flinger + dalvikvm
ios_runtime_manager.cpp	iOS/macOS	dyld_emulator + objc_runtime + macho_loader
harmony_runtime_manager.c	HarmonyOS	ability_runtime daemon + HAP install

5.3 Composants — Unified execution

Composant	Rôle	Lignes
filesystem_view.c	VFS union au-dessus de Linux /, Wine Z:/C:, Android /sdcard, iOS sandbox, HarmonyO	208
address_space.c	Registre mmap par runtime + partage cross-runtime via AsShare	247
network_stack.c	Network namespace par runtime + NAT + port forwarding	244
resource_manager.c	Moniteur 200ms + quotas CPU/mem/IO/FD/ports par runtime	223

5.4 Composants — Version management

Composant	Rôle	Lignes
version_registry.c	Registry JSON /var/lib/afros/runtimes.json + parser maison	255

Composant	Rôle	Lignes
update_checker.c	Poll manifest distant + compare semver	193
downloader.c	Download via curl/wget + vérification SHA-256	163
installer.c	Extraction archive + self-test + rollback	238

5.5 Composants — Selection + Monitoring

Composant	Rôle	Lignes
api_version.c	Entry points orchestrator_init/run_app/shutdown + afros_corebridge_api_version	261
selection_engine.c	Combine loader + registry + central_manager, compat score	220
monitoring.c	Watchdog 5s + sampling /proc/pid/statm + kill runtimes crash	273
resource_allocator.c	Pool 256 MiB + FD/port budgets per runtime + system caps	249
runtime_registry.c	Table runtimes actifs (handle → manager)	44
central_manager.c	orchestrator_monitor_system (Tier 2)	38

5.6 API publique gelée v1.0.0

L'API publique est gelée en v1.0.0 avec 3 stability tiers (voir document 3 section 4.2 et API.md dans le dépôt). Tier 1 Stable (breaking ⇒ MAJOR + RFC), Tier 2 Beta, Tier 3 Experimental.

```
// Entry points (Tier 1 Stable) int orchestrator_init(void); int
orchestrator_run_app(const char *path, const char *args); int
orchestrator_shutdown(void); const char *afros_corebridge_api_version(void); //
Loader API (Tier 1 Stable) app_type_t AppDetect(const char *path); format_info_t
FormatAnalyze(const char *path, app_type_t type); runtime_handle_t
IntelligentLoad(const char *path, const char *args); dep_list_t *ResolveDeps(const
char *path, app_type_t type); // Unified Execution API (Tier 1 Stable) vfs_view_t
*VfsCreateView(runtime_mask_t mask); as_region_t *AsReserve(runtime_handle_t rt,
void *hint, size_t len, int prot); int NetCreateNamespace(void); int
ResSetQuota(runtime_handle_t rt, const quota_t *q);
```

6. Couche 5 — Runtimes de compatibilité

Cette couche contient les 5 modules de compatibilité applicative. Chaque module wrappe un backend externe en exposant une API consommable par l'orchestrateur. Contient ~50 composants au total.

6.1 Module afros-winbridge (Wine)

Couche de compatibilité Windows basée sur Wine. 9 sous-systèmes, 36 fichiers, 6 160 lignes.

Sous-système	Composants	Rôle
pe_loader/	pe_parser, pe_to_elf, dll_resolver, resource_loader	PE/COFF parser + mapping + relocations + ressources
filesystem/	path_translator, drive_manager, file_attributes, fs_emulation	Traduction chemins + lettres lecteurs + NTFS ADS
registry/	registry_emulator, hive_manager, hive_to_file, software_hive, system_hive	Hives format binaire REGF/HBIN
syscall/	syscall_translator, io_manager, kernel_objects, nt_syscalls	Traduction NT syscalls (15 syscalls principaux)
server/	wineserver, process, registry	Socket Unix + table 256 process + cache 5s
services/	service_control_manager, windows_installer, msi_engine, task_scheduler	SCM + MSI + EVT + cron-like
com/	com_runtime, marshaler, proxy_stub, type_library, activation_context	CoInitialize + CoCreateInstance + marshaler
loader/	preloader, wine_loader	ELF preloader + wine_loader main
cache/	dll_cache, shader_cache, jit_cache, registry_cache	Caches LRU comptés + LRU

6.2 Module afros-androsandbox (Android)

Couche de compatibilité Android. 13 sous-systèmes, 41 fichiers, 7 332 lignes. Wraps ART + Binder + SurfaceFlinger.

Sous-système	Composants	Rôle
binder/	binder_driver, parcel, transaction, reference, service_manager	IPC kernel-like + servicemanager + 15 services défaut
dalvikvm/	dalvikvm.cc	CLI entry VM Dalvik (-cp, -Xmx, etc.)
art/runtime/	art_runtime, class_linker, jit_comlier	ArtRuntime singleton + heap + GC + JIT (mmap pages)
dex2oat/	dex2oat_main, compiler_driver	AOT compiler (DEX → OAT/ELF)
compiler/	optimizing_compiler, dex_to_native	Backends: SSA + regalloc ou dex→native fast path
surfaceflinger/	surface_flinger, layer, buffer_queue, hwcomposer_hal	Compositeur 60 Hz + drop-oldest ring + HWC HAL
framework/activity_manager/	activity_manager_service, activity_stack, task_record	AMS + ActivityStack + TaskRecord
framework/window_manager/	window_manager_service, display_content, window_state	WMS + DisplayContent + WindowState
framework/package_manager/	package_manager_service, package_parser, permission_manager	PMS + parsing XML + permissions runtime

Sous-système	Composants	Rôle
framework/content/	content_provider, content_resolver, uri_matcher	ContentProvider + ContentResolver + UriMatcher
framework/telephony/	telephony_manager, phone_interface	Telephony + Phone
services/	camera_service, audio_service, sensor_service, location_service	5 services (Vulkan, AAudio, sensors, GPS)
vfs/	android_fs, sdcard_emulation, ashmem_emulation	VFS Android (ashmem, /dev/binder, /system/fonts)

6.3 Module afros-incompat-engine (iOS/macOS via Darling)

Couche de compatibilité iOS/macOS. 11 sous-systèmes, 42 fichiers, 6 978 lignes. Wraps Mach-O loader + objc runtime + frameworks.

Sous-système	Composants	Rôle
macho_loader/	macho_parser, loader, symbol_resolver, dyld_emulation	Parse Mach-O, bind symbols, load dylibs
runtime/	objc_runtime, class_loader, message_dispatch	objc_msgSend + class registration + ARC
filesystem/	bundle_manager, apfs_emulation, hfsplus_driver	.app/.framework loader, APFS read-only + HFS+
sandbox/	ios_sandbox, container_manager, entitlements	Container isolation + entitlements + protection levels
codesign/	signature_verifier, certificate_chain, provisioning	CMS + chaîne Apple Root + .mobileprovision
frameworks/UIKIT/	UIApplication, UIView, UIWindow, UIViewController, UIControl	5 classes UIKit (.m)
frameworks/Foundation/	NSObject, NSString, NSArray, NSDictionary, NSData	5 classes Foundation (.m)
frameworks/AVFoundation/	AVPlayer, AVAsset, AVAudioSession	3 classes AV (.m)
frameworks/CoreAnimation/	CALayer, CAAnimation, CATransaction	3 classes CA (.m)
frameworks/CoreGraphics/	CGContext, CGPath, CGColor, CGImage	Quartz 2D (C, pas .m)
darling/src/	startup, darling_kernel	darling_init + mach_port_t/mach_msg emulation

6.4 Module afros-harmonygate (HarmonyOS)

Couche de compatibilité HarmonyOS. 9 sous-systèmes, 25 fichiers, 5 172 lignes. Wraps ability runtime + SoftBus distribué.

Sous-système	Composants	Rôle
distributed/device_manager/	device_discovery, device_monitor, capability_manager	Discovery mDNS + trusted/offline + pairing
distributed/data_sync/	distributed_data_mgr, sync_engine, version_control	Put/Get/Delete/Subscribe + vector clocks
distributed/softbus/discovery/	ble_discovery, mdns_discovery, coap_discovery	3 mécanismes discovery
distributed/softbus/connection/	tcp_connection, bluetooth_connection, wifi_direct	3 transports
distributed/softbus/authentication/	device_auth	HMAC challenge-response + session keys

Sous-système	Composants	Rôle
distributed/softbus/transmission/	file_transmission, stream_transmission	File chunked + stream low-latency
distributed/hardware_sharing/	camera_sharing, storage_sharing, sensor_sharing	Partage caméra/disk/sensor cross-device
ability/ability_runtime/	ability_lifecycle, ability_manager, ability_context	Lifecycle + AbilityManager + Context
ability/want/	want, want_params	Want (action/entity/uri) + WantParams (kv)

6.5 Runtime Linux natif

Linux natif ne constitue pas un module séparé : il est intégré directement dans linux_runtime_manager.c qui utilise clone() avec CLONE_NEWNS|CLONE_NEWPID pour sandboxing. Pas de wrapping nécessaire — Linux apps s'exécutent directement sur le noyau AfriOS.

7. Couche 6 — Graphismes (DXVK)

Couche de traduction DirectX → Vulkan. 7 sous-systèmes, 27 fichiers, 5 542 lignes. S'exécute comme un ensemble de DLLs Windows chargées par Wine.

7.1 Composants — D3D9 (DirectX 9)

Composant	Rôle	Lignes
d3d9.cpp	Direct3DCreate9 factory + IDirect3D9 impl	~80
d3d9_device.cpp	IDirect3DDevice9 (Clear, BeginScene, DrawPrimitive, SetRenderState)	~150
d3d9_shader.cpp	SM2/3 bytecode → SPIR-V translator skeleton	~100
d3d9_state.cpp	Render-state + sampler-state + viewport cache	~80
d3d9_swapchain.cpp	IDirect3DSwapChain9 + Present → vkQueuePresentKHR	~80

7.2 Composants — D3D11 (DirectX 11)

Composant	Rôle	Lignes
d3d11_device.cpp	ID3D11Device (CreateBuffer, CreateTexture2D, CreateVertexShader, ...)	~180
d3d11_context.cpp	ID3D11DeviceContext immediate (VSSetShader, DrawIndexed, OMSetRenderTargets, ...	~200
d3d11_buffer.cpp	ID3D11Buffer wraps VkBuffer + Map/Unmap	~80
d3d11_texture.cpp	ID3D11Texture2D wraps VkImage + subresource access	~100
d3d11_shader.cpp	HLSL → SPIR-V via glslang	~80

7.3 Composants — D3D12 (DirectX 12)

Composant	Rôle	Lignes
d3d12_command_list.cpp	ID3D12GraphicsCommandList (DrawInstanced, ResourceBarrier, ...)	~180
d3d12_resource.cpp	ID3D12Resource wraps VkImage/VkBuffer + state tracking	~120
d3d12_description_heap.cpp	ID3D12DescriptorHeap → VkDescriptorPool + sets	~100
d3d12_device.app	ID3D12Device (CreateCommandQueue, CreateDescriptorHeap, ...)	~120

7.4 Composants — DXGI + HLSL + Vulkan + Utils

Sous-système	Composants	Rôle
dxgi/	dxgi_factory, dxgi_adapter, dxgi_swapchain	IDXGIFactory + IDXGIAdapter + IDXGISwapChain

Sous-système	Composants	Rôle
hlsl/	hlsl_compiler, hlsl_optimizer, spirv_generator	Lexer + parser + DCE + SPIR-V emitter 1.3
vulkan/	vulkan_device, vulkan_memory, vulkan_pipeline, vulkan_presenter	VkDevice + VMA + pipeline cache + swapchain
util/	cache_manager, shader_cache, performance_hud	LRU disk + RAM + FPS overlay

7.5 Points d'entrée Wine

DXVK exporte ses points d'entrée en extern "C" pour permettre le chargement via dlopen par Wine. Le binaire produit est libafros-dxvk.so + 4 symlinks Wine DLL (d3d9.dll.so, d3d11.dll.so, d3d12.dll.so, dxgi.dll.so).

```
extern "C" IDirect3D9* Direct3DCreate9(UINT sdk); extern "C" IDirect3D9* Direct3DCreate9Ex(UINT sdk, void**); extern "C" HRESULT CreateDXGIFactory(REFIID riid, void **factory); extern "C" HRESULT CreateDXGIFactory1(REFIID riid, void **factory); extern "C" HRESULT D3D11CreateDevice(IDXGIAdapter*, D3D_DRIVER_TYPE, HMODULE, UINT, const D3D_FEATURE_LEVEL*, UINT, UINT, ID3D11Device**, D3D_FEATURE_LEVEL*, ID3D11DeviceContext**); extern "C" HRESULT D3D12CreateDevice(IUnknown*, D3D_FEATURE_LEVEL, REFIID, void**);
```

8. Vue transversale des interconnexions

Cette section montre comment les composants des 6 couches collaborent lors d'un scénario concret : un utilisateur lance notepad.exe. Le flux traverse les 6 couches de haut en bas, chaque composant jouant un rôle précis.

8.1 Scénario : lancement de notepad.exe

```
[User] afros-launch notepad.exe | v Couche 4: orchestrator_run_app(path, args) | (a) app_detector.c: magic MZ -> APP_TYPE_WINDOWS_PE | (b) format_analyzer.c: subsystem=GUI, arch=x86_64 | (c) selection_engine.c: WinRuntimeManager (compat score) | (d) intelligent_loader.c: cache LRU hit | (e) dependency_resolver.c: kernel32.dll, user32.dll v Couche 5: WinRuntimeSpawn -> win_runtime_manager.c | (a) wineserver.c: démarre socket Unix | (b) wine_loader.c: init registry, filesystem, COM | (c) syscall_translator.c: branche NT -> Linux syscalls | (d) pe_parser.c: mappe notepad.exe en mémoire | (e) dll_resolver.c: kernel32 -> libafros-winbridge.so v Couche 6: DXVK (si appel graphique) | (a) notepad invoque GDI/Direct2D | (b) d3d11.dll.so (symlink) intercepte l'appel | (c) d3d11_device.cpp + d3d11_context.cpp traduisent | (d) vulkan_device.cpp émet vkCmdBind* + vkCmdDraw | (e) vulkan_presenter.cpp présente via vkQueuePresentKHR v Couche 3: Noyau (intégration silencieuse) | (a) afros_cfs.c: place wineserver thread sur big core | (b) filesystem_view.c (orchestrateur): C:\Users -> /home | (c) power_aware_sched.c: notepad = USER_VISIBLE | (d) solar_aware.c: si batterie < 25%, gel UI non critical v Couche 2: Firmware (déjà booté, passif) | (a) MemoryTest.c: a validé RAM au boot | (b) ABSlotManager.c: slot A actif, boot_success=1 | (c) SecureBootPolicy.c: a vérifié signatures kernel v Couche 1: Matériel | (a) CPU x86_64: exécute les instructions | (b) MMU: traduit virtuel -> physique | (c) APIC: distribue IRQs | (d) VGA/GPU: rafraîchit l'écran v SORTIE: fenêtre notepad affichée
```

8.2 Composants impliqués par couche

Couche	Composants impliqués	Nombre
1. Matériel	CPU, MMU, APIC, GPU	4
2. Firmware	MemoryTest, ABSlotManager, SecureBootPolicy	3 (passifs)
3. Noyau	afros_cfs, filesystem_view (orchestrateur), power_aware_sched, solar_aware	4
4. Orchestrateur	app_detector, format_analyzer, selection_engine, intelligent_loader, dependency_resolver	5
5. Runtime Win	wineserver, wine_loader, syscall_translator, pe_parser, dll_resolver	5
6. DXVK	d3d11.dll.so, d3d11_device, d3d11_context, vulkan_device, vulkan_presenter	5
Total	26 composants impliqués pour un lancement	~17%

Sur les ~150 composants d'AfriOS, environ 26 (17 %) sont impliqués dans un simple lancement d'application. Les autres composants sont soit passifs (firmware déjà booté), soit en attente (autres runtimes, OTA, Diagnostics, etc.) soit activés sur d'autres scénarios (boot, mise à jour, debug).

8.3 Dépendances critiques amont/aval

Le tableau ci-dessous liste les dépendances critiques entre composants de couches différentes. Une cassure de l'une de ces dépendances bloque le scénario.

Dépendance	Bloque	Type
Couche 1 → Couche 2 (boot vector)	Boot complet	Matériel
Couche 2 → Couche 3 (kernel_main)	Démarrage noyau	Firmware→Noyau
Couche 3 → Couche 4 (HAL ops)	Orchestrateur	Noyau→Orch.
Couche 4 → Couche 5 (runtime_ops_t)	Tous runtimes	Orch.→Runtimes
Couche 5 → Couche 6 (DXVK aliases)	Apps graphiques Windows	Wine→DXVK
Couche 4 → Couche 5 (loader→manager)	Lancement apps	Orch.→Runtimes

9. Conclusion

Ce document a présenté les ~150 composants qui composent AfriOS, répartis sur 6 couches. Pour chaque couche, nous avons listé les composants avec leur rôle, leurs fichiers sources, et l'API qu'ils exposent pour la couche supérieure. La granularité adoptée — composant, ni vision globale (document 2) ni fichier (document 3) — permet de comprendre l'architecture sans se perdre dans les détails.

Les composants les plus critiques sont : (1) app_detector et selection_engine dans l'orchestrateur — sans eux, aucun lancement d'app ; (2) hal_init et les 5 _port.c console par arch — sans eux, pas de boot noyau ; (3) wineserver et syscall_translator dans winbridge — sans eux, pas d'apps Windows ; (4) ABSlotManager dans le firmware — sans lui, pas de mise à jour A/B robuste.

Pour approfondir un composant précis, consultez : le document 3 Modules pour le détail des fichiers ; le white paper Optimisations africain pour les algorithmes des modules réseau/power ; le API.md de CoreBridge pour la spec formelle de l'API orchestrateur ; le code source lui-même pour les détails d'implémentation (chaque fichier a un header /** @file */ et chaque fonction a un doc comment).

Pour ajouter un nouveau composant : (1) identifier la couche appropriée ; (2) suivre les conventions de nommage (op-table C, vtable C++, ou protocole UEFI) ; (3) exposer l'API via un header dans le include/ du module ; (4) implémenter dans src/ ; (5) ajouter au CMakeLists.txt du module ; (6) ajouter des tests dans le module approprié ; (7) documenter dans ce document et dans le worklog.